

CHAPTER – 1

INTRODUCTION

The stock market plays a significant role in the global financial system by enabling capital formation, investment opportunities, and economic growth. Stock prices reflect the performance, stability, and future expectations of publicly traded companies. Investors, traders, and financial institutions continuously analyse stock price movements to make informed decisions regarding buying, selling, and portfolio management. However, predicting stock prices remains a challenging task due to the dynamic, nonlinear, and highly volatile nature of financial markets. Market prices are influenced by various internal and external factors such as company performance, macroeconomic indicators, global events, and investor sentiment. These factors introduce uncertainty and complexity into stock price movements, making accurate forecasting difficult.

The stock market plays a significant role in the global financial system by enabling capital formation, investment opportunities, and economic growth. Stock prices reflect the performance, stability, and future expectations of publicly traded companies. Investors, traders, and financial institutions continuously analyse stock price movements to make informed decisions regarding buying, selling, and portfolio management. However, predicting stock prices remains a challenging task due to the dynamic, nonlinear, and highly volatile nature of financial markets. Market prices are influenced by various internal and external factors such as company performance, macroeconomic indicators, global events, and investor sentiment. These factors introduce uncertainty and complexity into stock price movements, making accurate forecasting difficult.

In recent years, machine learning and deep learning techniques have gained significant attention in the field of financial forecasting. Deep learning models are particularly effective for sequential data because they can automatically learn complex patterns without requiring manual feature engineering. Recurrent Neural Networks (RNN) are designed to process sequential inputs by maintaining hidden states that carry information across time steps. However, traditional RNNs suffer from limitations such as the vanishing gradient problem, which restricts their ability to learn long-term dependencies.

To overcome these limitations, advanced recurrent architectures such as Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU) were developed. LSTM networks use memory cells and gating mechanisms to selectively retain and forget information, enabling them to model long-term dependencies effectively. GRU networks provide a simplified version of LSTM with fewer parameters while maintaining competitive performance. Both LSTM and

GRU have been widely applied in financial time series forecasting and have demonstrated improved prediction accuracy compared to traditional models.

In addition to recurrent architectures, Convolutional Neural Networks (CNN), traditionally used for image processing, have also been adapted for time series analysis. CNN models are capable of extracting local features and short-term patterns from sequential data using convolutional filters. When applied to stock price sequences, CNN models can identify short-term trends and fluctuations that may contribute to price prediction. Furthermore, hybrid models such as CNN-LSTM combine the strengths of both architectures, where CNN layers extract relevant features and LSTM layers capture long-term temporal dependencies. Such hybrid models often demonstrate superior performance by learning both local and global patterns in financial time series data.

This study focuses on developing and comparing multiple deep learning models for stock price forecasting. Five models are implemented and evaluated: Recurrent Neural Network (RNN), Gated Recurrent Unit (GRU), Long Short-Term Memory (LSTM), Convolutional Neural Network (CNN), and a hybrid CNN-LSTM model. The objective is to analyse their performance and identify the most suitable architecture for forecasting stock prices and returns. To ensure realistic and reliable evaluation, walk-forward validation is employed. Unlike random data splitting, walk-forward validation preserves the chronological order of time series data and simulates real-world forecasting conditions, making performance evaluation more practical and reliable.

The dataset used in this study consists of historical weekly stock price data of Infosys Limited, one of the leading information technology companies in India. Infosys was selected due to its strong market presence, consistent financial performance, and availability of long-term historical data. The stock exhibits clear trends, volatility patterns, and time-dependent behaviour, making it an appropriate candidate for time series forecasting analysis. The dataset includes features such as Open, High, Low, Close, and Volume, which provide comprehensive information about price movements and trading activity. Descriptive statistical analysis and exploratory data analysis are conducted to understand the underlying structure and behaviour of the dataset before model development.

Feature engineering techniques, including candlestick-based variable creation, were initially explored to enhance model performance. However, experimental evaluation showed that these additional features did not significantly improve prediction accuracy and, in some cases,

introduced noise. Therefore, the final models were trained using the core price-based features to ensure model stability and reliability. Data preprocessing steps such as normalization and sequence creation were performed to prepare the dataset for deep learning models.

This research contributes to the growing body of work on deep learning-based financial forecasting by providing a systematic comparison of multiple architectures under a unified experimental framework. By incorporating walk-forward validation and evaluating different model structures, the study aims to provide insights into the effectiveness of deep learning approaches for stock price prediction. The findings of this study may assist investors, researchers, and practitioners in understanding how deep learning models can be applied to financial time series forecasting and how different architectures perform under realistic market conditions.

CHAPTER – 2

LITERATURE REVIEW

Stock price forecasting has evolved from traditional statistical techniques to advanced deep learning approaches. This chapter reviews key studies related to time series forecasting, deep learning models, hybrid architectures, and validation techniques relevant to this research.

[1] Comprehensive Review of Data-Driven Neural Networks says that, Bao et al. (2025) provide a comprehensive review of data-driven neural network methods in stock forecasting, covering developments from 2015 to 2023. The study discusses classic and emerging architectures including RNNs, CNNs, Transformers, Graph Neural Networks, GANs, and large language models, and highlights remaining challenges and future directions for stock market forecasting.

[2] Stock Market Prediction Using Deep Learning and Monte Carlo Simulation says that Karunamurthy et al. (2024) proposed a deep learning framework that combines LSTM, GRU, CNN, and Monte Carlo simulation to handle uncertainty and volatility in stock price prediction. The authors show that combining these techniques provides more robust forecasts across different market conditions and datasets.

[3] Hybrid Ensemble Frameworks says that Nugroho and colleagues (2025) introduced a hybrid ensemble model that integrates ARIMA (for linear patterns), Random Forest (for nonlinear dependencies), and LSTM for temporal patterns. The ensemble approach significantly outperformed standalone models across multiple stock datasets, indicating the usefulness of combining statistical and deep learning techniques.

[4] Deep Learning-Based Financial Forecasting Review says that Giantsidi (2025) conducted a thorough review of deep learning applications in financial forecasting, summarizing 187 Scopus-indexed studies from 2020–2024. The review highlights various deep learning algorithms and identifies key gaps in research, such as model interpretability and real-time forecasting challenges.

[5] Comparative Study of Deep Learning Models for Indian Market says that Barua et al. (2024) performed a comparative analysis of RNN, LSTM, CNN, GRU, and Attention LSTM on stock price prediction for multiple Indian companies (e.g., HDFC, TCS, ICICI) and the Nifty 50 index. The study found distinct strengths among architectures, with sequence models showing better temporal pattern learning.

[6] LSTM-Centric Forecasting Across Major Technology Stocks (2024) Li et al. (2023/2024) investigated the application of LSTM networks on major tech stock prices (e.g., Apple, Google,

Microsoft), demonstrating improved accuracy over traditional regression techniques. Their study confirms the effectiveness of LSTM for long-term dependencies in financial time series.

[7] Temporal Gated Recurrent Neural Networks with Bayesian Optimization Dinda (2024) developed a gating recurrent neural network enhanced with Tree-structured Parzen Estimator (TPE) for hyperparameter optimization. This approach improves performance for stock index forecasting by adapting model configurations, reducing prediction errors such as MAPE and RMSE compared to baseline models.

[8] Hybrid LSTM-SHO Model says that Gülmez (2025) proposed the LSTM-SHO hybrid model that combines LSTM with Seahorse Optimization algorithm. This hybrid model enhanced stock prediction accuracy and provided better performance than traditional LSTM and other optimization-based variants, showing the benefit of bio-inspired optimization.

[9] Transformer-Style Neural Networks for Stock Forecasting says that Szydłowski and Chudziak (2024) investigated Transformer-style architectures called Hidformer for stock price prediction. These models adapt transformer design for sequential financial data, highlighting the potential of attention-based methods for capturing long-term dependencies beyond standard recurrent networks.

[10] Deep Learning for Financial Trend Prediction says that Radfar et al. (2025) explored the feasibility of deep learning models for stock trend prediction and technical analysis. Their study discussed classical and deep learning methods applied to real market behavior, emphasizing how modern neural architectures can outperform traditional models in forecasting accuracy.

CHAPTER – 3

METHODOLOGY

This chapter describes the methodology used to develop the stock price forecasting system. It explains the dataset used in the study, preprocessing techniques, feature engineering process, model development, and evaluation methods used to compare different deep learning models.

3.1 Dataset Description

The dataset used in this study consists of historical stock price data of Infosys Limited, one of the leading information technology companies in India. Infosys is widely recognized for its strong global presence and stable financial performance in the IT services sector. The company is publicly traded on major stock exchanges and its stock price data is frequently analysed by investors and financial analysts. Due to its strong market position, availability of long-term historical data, and observable price trends, the Infosys stock dataset was selected for this research.

The dataset was obtained from Yahoo Finance, a widely used financial data platform that provides reliable and publicly accessible historical stock market data. Yahoo Finance offers detailed information about stock prices and trading activity, making it an appropriate source for financial time series analysis and forecasting research.

The dataset used in this study contains weekly stock price observations covering the period from 2011 to 2026, with approximately 780 observations. Weekly data was selected to reduce short-term fluctuations and noise. Using weekly observations allows the forecasting models to focus more effectively on long-term price trends and market patterns.

The structure of the dataset and the variables used in this study are presented in Table 3.1.

Variable	Description
Date	Represents the trading week date
Open	Opening price of the stock at the beginning of the week
High	Highest price reached during the week
Low	Lowest price recorded during the week
Close	Closing price of the stock at the end of the week
Volume	Total number of shares traded during the week

Table 3.1: Description of Dataset Variables

The Open, High, Low, and Close (OHLC) values represent the major price indicators that describe the movement of the stock during the trading period. These indicators are commonly used in financial market analysis to understand stock price trends and patterns. The Volume

variable represents the number of shares traded during the week and reflects the level of trading activity in the market.

To better understand the structure of the dataset, a sample of the dataset records is presented in Table 3.2.

Date	Open	High	Low	Close	Volume
31-01-2011	264.2423401	270.8139412	262.3783647	268.6725411	6959880
07-02-2011	263.6311035	273.9349715	261.853825	266.5831029	40335568
14-02-2011	268.7852478	272.6605783	263.6007912	263.9042291	24994672
21-02-2011	260.7224731	275.5042312	257.1419023	268.7592371	49414224
28-02-2011	265.4820862	271.7762622	257.2632516	261.9968902	45234872

Table 3.2: Sample Records from the Infosys Stock Dataset

These variables provide essential information about stock price behaviour and trading activity over time.

3.2 Feature Engineering

Feature engineering is an important step in preparing data for machine learning and deep learning models. In this study, candlestick pattern analysis was used to create additional variables from the stock price data.

Candlestick patterns are commonly used in financial market analysis to identify price movements and potential trend changes. These patterns are derived from the Open, High, Low, and Close (OHLC) values of the stock price. The objective of creating candlestick-based variables was to capture potential price behaviour patterns that could improve the forecasting performance of the models.

Using the OHLC data, several candlestick-based variables were generated to represent price movement characteristics. These additional variables were incorporated into the dataset and tested during the model training phase to evaluate their impact on prediction performance.

However, after experimental evaluation, it was observed that the candlestick-based variables did not significantly improve the forecasting accuracy of the models. In some cases, these variables introduced additional noise into the dataset and negatively affected model performance. Therefore, the candlestick-derived variables were removed, and the final models were trained using the original stock price features.

This feature engineering process helped ensure that only the most relevant variables were retained for the forecasting models.

3.3 Data Preprocessing

Data preprocessing is a crucial step in preparing the dataset for machine learning and deep learning models. Raw financial datasets often contain inconsistencies, formatting issues, or variables that require transformation before they can be used for modelling. Proper preprocessing ensures that the dataset is clean, structured, and suitable for time series forecasting.

In this study, several preprocessing steps were applied to prepare the stock price dataset before training the deep learning models.

3.3.1 Data Cleaning

The first step in preprocessing involved examining the dataset for missing or inconsistent values. Missing data can negatively affect model training and prediction performance. Therefore, the dataset was carefully inspected to ensure data completeness and consistency.

The dataset used in this study did not contain any significant missing values. As a result, no data imputation techniques were required. The dataset was considered clean and suitable for further processing.

3.3.2 Date Formatting and Chronological Ordering

Since the dataset represents time series data, maintaining the correct chronological order is essential. The Date column was converted into a proper datetime format to ensure accurate time representation.

After converting the date format, the dataset was sorted in chronological order based on the date variable. Maintaining the correct time order ensures that historical observations appear before future observations, which is necessary for proper time series forecasting.

3.3.3 Feature Selection

After organizing the dataset, the relevant variables required for forecasting were selected. The final dataset used for model development included the following variables:

- Open
- High

- Low
- Close
- Volume

These variables provide essential information about stock price movement and trading activity. The Close price was considered the target variable for forecasting because it represents the final trading price of the stock during the given time period.

3.3.4 Data Normalization

Deep learning models perform more effectively when input data is scaled within a consistent range. Large variations in feature values can slow down the training process and reduce model performance.

To address this issue, the Min-Max normalization technique was applied to scale all input features between 0 and 1. This transformation ensures that each variable contributes equally to the learning process.

The Min-Max normalization formula is given as:

$$X_{Scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

where

X represents the original value,

X_min represents the minimum value of the variable, and

X_max represents the maximum value of the variable.

Applying normalization helps improve model convergence during training and enhances prediction performance.

After completing the preprocessing steps, the dataset was properly cleaned, formatted, and scaled. The processed dataset was then used for sequence creation and model training in the next stage of the methodology.

3.4 Time Series Sequence Creation

Time series forecasting models require sequential input data in order to learn patterns from past observations and predict future values. Unlike traditional machine learning models, deep learning architectures such as Recurrent Neural Networks (RNN), Gated Recurrent Units

(GRU), and Long Short-Term Memory (LSTM) are specifically designed to capture temporal dependencies in sequential datasets. Therefore, the dataset must be transformed into sequences before it can be used for model training.

In this study, a sliding window approach was used to convert the stock price data into input sequences suitable for deep learning models.

3.4.1 Sliding Window Method

The sliding window method is commonly used in time series forecasting to create input-output pairs from historical data. In this approach, a fixed number of previous observations are used as input to predict the next value in the sequence.

For example, if the sequence length is defined as n , then the model uses the previous n time steps to predict the value at time $t+1$. This allows the model to learn patterns and relationships between past stock prices and future price movements.

The sliding window process continues across the entire dataset, generating multiple input sequences for training the models.

3.4.2 Input Sequence Length

The number of previous observations used for prediction is referred to as the sequence length or time step. In this study, a fixed sequence length was selected to represent historical stock price movements.

Each input sequence consists of historical values of the selected features (Open, High, Low, Close, and Volume), while the corresponding output represents the next closing price of the stock.

This structure enables the deep learning models to capture temporal relationships in stock price movements and learn patterns over time.

3.4.3 Input and Output Structure

After applying the sliding window method, the dataset was transformed into two components:

- Input sequences (X)
- Target variable (y)

The input sequences consist of historical observations used for training the models, while the target variable represents the value to be predicted.

The structure of the dataset after sequence creation can be represented as follows:

$X = [\text{previous stock price observations}]$

$y = [\text{next closing price}]$

Each input sequence contains multiple time steps and features, while the output represents the predicted closing price for the next time step.

3.4.4 Dataset Transformation for Deep Learning Models

After generating the sequences, the dataset was reshaped into a three-dimensional structure suitable for deep learning models. The final input structure can be represented as:

[Samples, Time Steps, Features]

where:

- Samples represent the total number of training observations
- Time Steps represent the sequence length
- Features represent the number of input variables (Open, High, Low, Close, Volume)

This structure enables deep learning models to process sequential information effectively and learn temporal dependencies from the dataset.

After transforming the dataset into sequential format, the data was divided into training and testing sets for model development and evaluation.

3.5 Train–Test Split

In time series forecasting, it is important to divide the dataset into separate training and testing sets in order to evaluate the predictive performance of the models. The training dataset is used to train the forecasting models and learn patterns from historical data, while the testing dataset is used to evaluate how well the trained models perform on unseen data.

Unlike traditional machine learning tasks where data can be randomly split, time series data must be divided while preserving the chronological order. This ensures that past observations are used to predict future values, which reflects real-world forecasting conditions.

3.5.1 Training Dataset

The training dataset consists of the initial portion of the time series data and is used to train the deep learning models. In this study, approximately 80% of the dataset was used for training.

The training data allows the models to learn temporal patterns, trends, and relationships present in historical stock price movements. During this stage, the models adjust their internal parameters in order to minimize prediction errors.

3.5.3 Chronological Data Splitting

Since the dataset represents sequential financial data, the split between training and testing sets was performed chronologically. This means that earlier observations were used for training, while later observations were used for testing.

Maintaining chronological order is essential in time series forecasting because random splitting could introduce future information into the training data, which would lead to unrealistic prediction performance.

The dataset splitting process is illustrated conceptually in **Figure 3.1**.

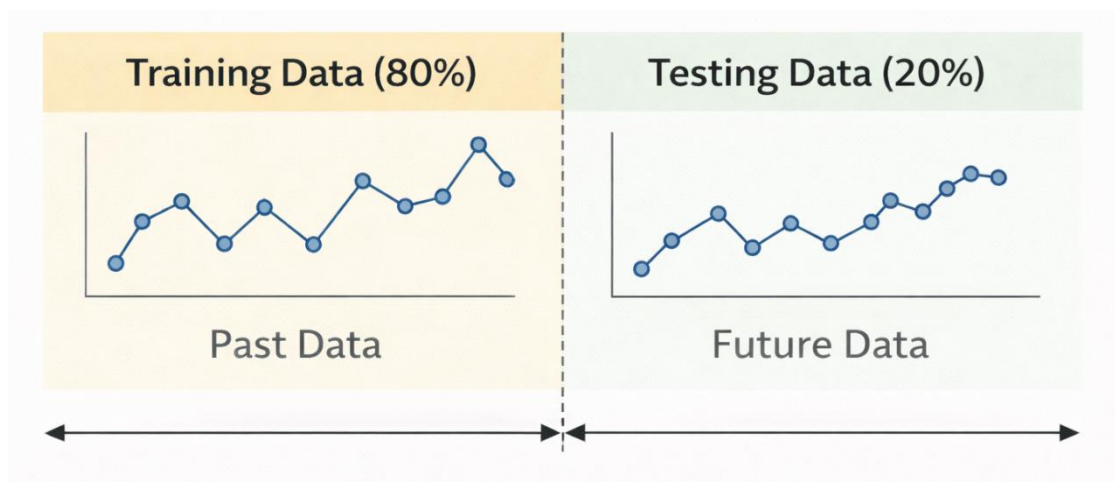


Figure 3.1: Train-Test Split of Time Series Data

This approach ensures that the models are trained using historical observations and evaluated on future data, providing a realistic assessment of forecasting performance.

After splitting the dataset into training and testing sets, the models were evaluated using walk-forward validation, which simulates real-world forecasting conditions by updating the training data sequentially.

3.6 Walk-Forward Validation

Walk-forward validation is a widely used evaluation technique for time series forecasting models. Unlike traditional validation methods that randomly split the dataset, walk-forward validation preserves the chronological order of observations and evaluates the model using a realistic forecasting approach. This method simulates how forecasting models are used in real-world financial markets, where future values are predicted based on past observations.

In this study, walk-forward validation was applied to evaluate the performance of the deep learning models developed for stock price forecasting. The method ensures that the models are trained only on historical data and then used to predict future stock prices sequentially.

3.6.1 Concept of Walk-Forward Validation

In walk-forward validation, the model is initially trained using the training dataset. After training, the model generates a prediction for the next time step in the testing dataset. Once the prediction is made, the actual observed value from the testing dataset is added to the training dataset. The model then moves forward and generates the next prediction.

This process continues sequentially until predictions are generated for the entire testing dataset. The step-by-step procedure can be summarized as follows:

1. Train the model using the training dataset.
2. Predict the next value in the testing dataset.
3. Add the actual observed value to the training dataset.
4. Retrain or update the model if necessary.
5. Repeat the process until all test observations are predicted.

This sequential forecasting approach allows the model to continuously update its knowledge as new observations become available.



Figure 3.2: Walk Forward Validation

3.6.2 Advantages of Walk-Forward Validation

Walk-forward validation provides several advantages when evaluating time series forecasting models:

- It preserves the chronological order of data.
- It prevents data leakage from future observations.
- It provides a realistic simulation of real-world forecasting scenarios.
- It allows models to be evaluated on multiple forecasting steps.

Due to these advantages, walk-forward validation is widely recommended for financial time series forecasting studies.

3.6.3 Walk-Forward Validation Process

The walk-forward validation process used in this study is illustrated in Figure 3.3.

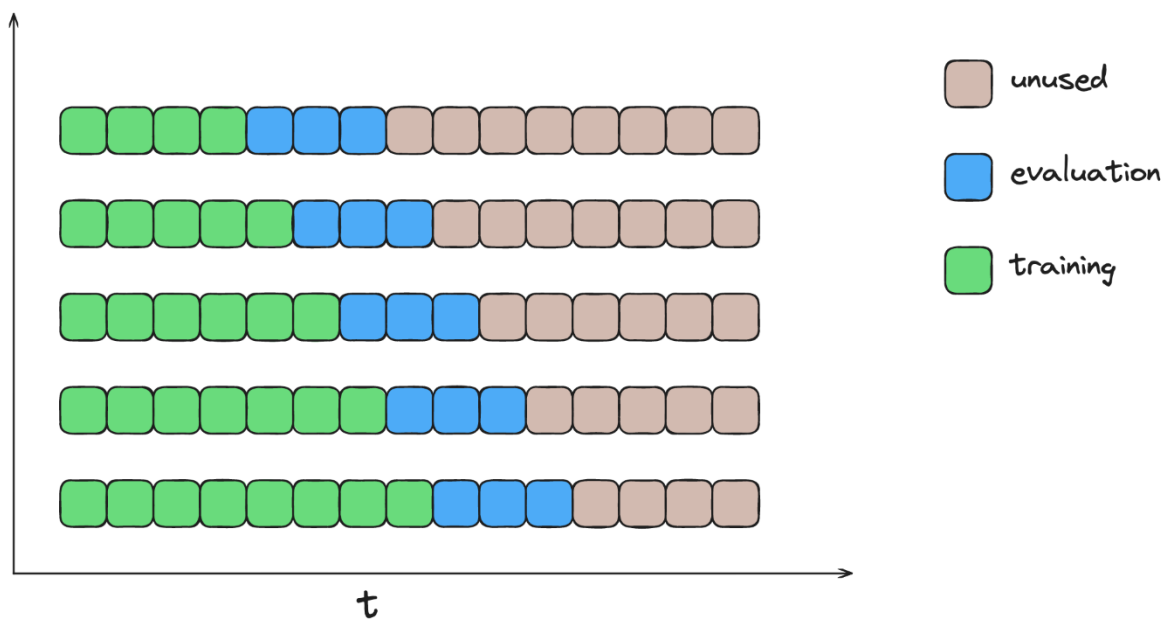


Figure 3.3: Walk-Forward Validation Process

Initially, the model is trained using the available historical data. The model then predicts the next stock price. After the prediction is made, the actual observed value is incorporated into the training dataset, and the process continues sequentially across the testing dataset.

This method ensures that the forecasting models are evaluated under realistic conditions similar to real-world stock price prediction scenarios.

3.7 Deep Learning Models

Deep learning models have gained significant attention in time series forecasting due to their ability to capture complex nonlinear relationships and temporal dependencies in sequential data. Unlike traditional statistical models, deep learning architectures can automatically learn patterns from historical data without requiring extensive manual feature engineering.

In this study, several deep learning models were implemented to forecast stock prices using historical time series data. These models include Recurrent Neural Network (RNN), Gated Recurrent Unit (GRU), Long Short-Term Memory (LSTM), Convolutional Neural Network (CNN), and a hybrid CNN-LSTM model. Each model has unique characteristics and advantages in capturing different patterns within time series data.

The objective of using multiple models is to compare their forecasting performance and identify the most suitable architecture for predicting stock prices and returns.

3.7.1 Recurrent Neural Networks (RNN)

Recurrent Neural Networks (RNNs) are a class of neural networks designed to process sequential data by retaining information from previous steps. They are especially effective for tasks where context and order matter.

- Designed for sequential and temporal data
- Maintains memory of past inputs
- Widely used in NLP, forecasting and speech tasks

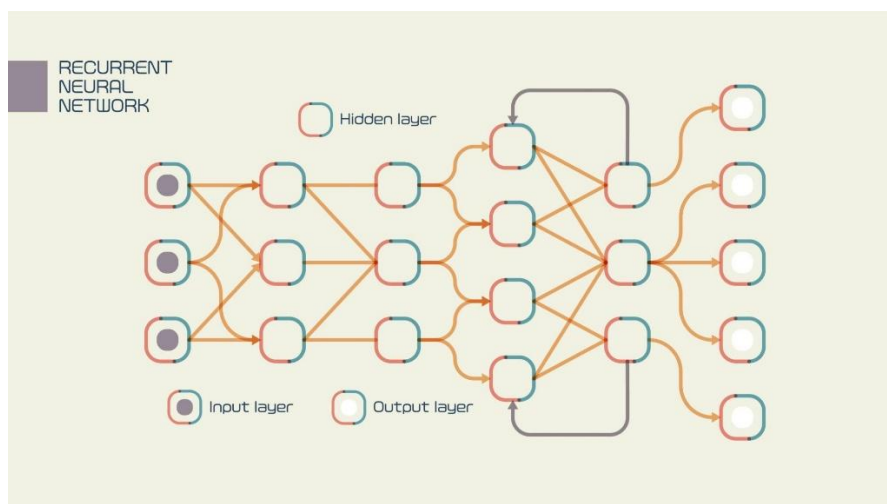


Figure 3.4: Recurrent Neural Networks (RNN)

3.7.1.2 Components of RNNs

There are mainly two components of RNNs,

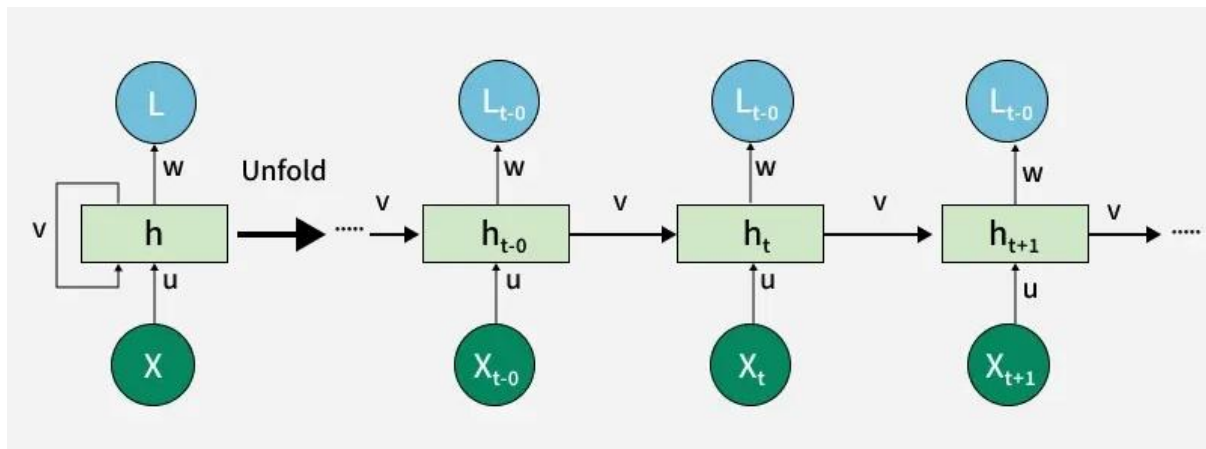


Figure 3.5: Components of Recurrent Neural Networks (RNN)

1. Recurrent Neurons

The fundamental processing unit in RNN is a Recurrent Unit. They hold a hidden state that maintains information about previous inputs in a sequence. Recurrent units can "remember" information from prior steps by feeding back their hidden state, allowing them to capture dependencies across time.

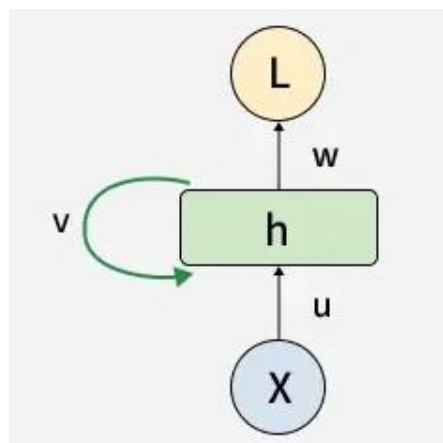


Figure 3.6: Recurrent Neurons

2. RNN Unfolding

RNN unfolding or unrolling is the process of expanding the recurrent structure over time steps. During unfolding each step of the sequence is represented as a separate layer in a series illustrating how information flows across each time step.

This unrolling enables backpropagation through time (BPTT) a learning process where errors are propagated across time steps to adjust the network's weights enhancing the RNN's ability to learn dependencies within sequential data.

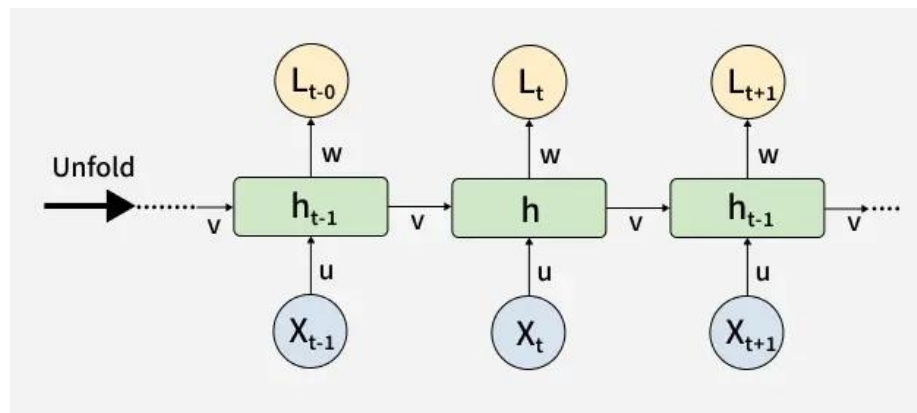


Figure 3.7: RNN Unfolding

3.7.1.2 Structure of RNN

The structure of a Recurrent Neural Network (RNN) is designed to process sequential or time-series data.

Unlike traditional neural networks, RNN has connections that loop back, allowing the network to store information from previous steps.

An RNN mainly consists of three main components:

1. Input Layer
2. Hidden (Recurrent) Layer
3. Output Layer

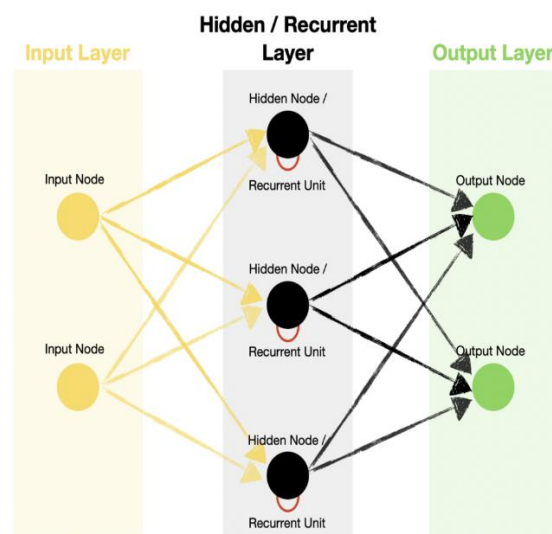


Figure 3.8: Structure of RNN

1. Input layer:

The input layer receives the sequential data and feeds it into the network one element at a time. Each input in the sequence is represented as x_t , where t denotes the time step. The input layer passes this information to the hidden layer for further processing.

2. Hidden (Recurrent) Layer

The hidden layer, also known as the recurrent layer, is the core component of the RNN. It processes the current input along with information from the previous time step through a hidden state. This hidden state enables the network to retain memory of past inputs, allowing it to capture temporal dependencies in the data. The hidden layer combines current information + past memory.

The hidden state at time t is computed using the following equation:

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

Where:

- x_t = current input
- h_{t-1} = previous hidden state (memory)
- h_t = current hidden state
- W_x = weight for input
- W_h = weight for previous hidden state
- b = bias
- f = activation function (tanh or ReLU)

3. Output Layer

The output layer produces the final prediction of the network based on the hidden state. The output at time t is calculated as:

$$y_t = W_y h_t + b$$

where:

- h_t is the hidden state,

- W_y is the weight matrix for the output layer,
- y_t represents the predicted output.

The output may be generated at each time step or only at the final time step, depending on the task.

3.7.2 Gated Recurrent Unit (GRU)

Gated Recurrent Unit (GRU) networks are a type of recurrent neural network designed to handle sequential data while reducing the complexity of traditional RNNs. GRUs are a simplified advancement of LSTM, where they merge multiple gates into update and reset gates, hence learning long-term dependencies with faster training and fewer parameters.

- Simplified alternative to LSTM
- Handles sequence and time-series data effectively
- Widely used in NLP, speech and forecasting tasks

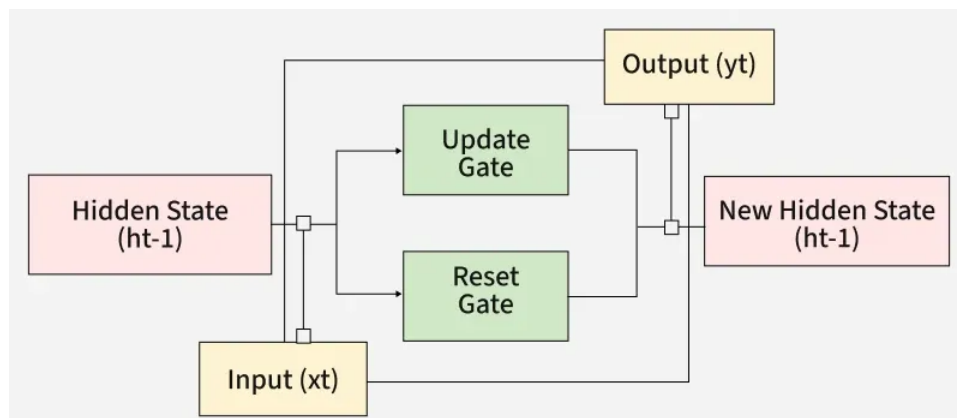


Figure 3.9: Structure of GRU-1

3.7.2.1 Structure of GRU

The structure of a GRU consists of an input vector, hidden state, and gating mechanisms that regulate the flow of information through the network.

A GRU unit primarily contains two gates: the update gate and the reset gate, which control how information is stored, updated, and forgotten in the hidden state which is shown in figure 3.9

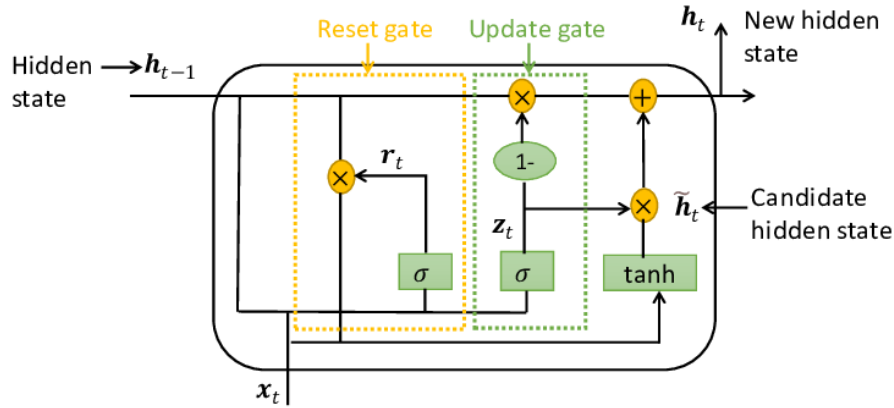


Figure 3.10: Structure of GRU-2

1. Input Layer

The input layer receives the input sequence x_t at time step t . This input represents the current data point in the sequence. The input is combined with the hidden state from the previous time step to determine how the network updates its memory.

2. Hidden State

The hidden state h_t represents the memory of the GRU. It carries information from previous time steps and is updated continuously as new inputs are processed. The hidden state allows the network to maintain contextual information across the sequence.

3. Update Gate

The update gate z_t determines how much of the previous hidden state should be retained and how much new information should be incorporated into the current hidden state.

The update gate is calculated as:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

where:

- x_t is the current input,
- h_{t-1} is the previous hidden state,
- W_z and U_z are weight matrices,
- b_z is the bias term,
- σ represents the sigmoid activation function.

The sigmoid function produces values between 0 and 1, which control the proportion of previous information retained.

4. Reset Gate

The reset gate r_t determines how much of the previous hidden state should be ignored when computing the candidate hidden state. It allows the model to discard irrelevant past information.

The reset gate is computed as:

where W_r , U_r , and b_r represent the weights and bias associated with the reset gate.

5. Candidate Hidden State

The candidate hidden state $\tilde{h}_t$ represents the new content that could be added to the hidden state. It is calculated using the reset gate to control the influence of the previous hidden state.

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$$

Here, $\odot$ denotes element-wise multiplication.

6. Final Hidden State

The final hidden state h_t is computed by combining the previous hidden state and the candidate hidden state, controlled by the update gate.

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

This mechanism enables the GRU to adaptively retain past information or incorporate new information, depending on the relevance of the current input.

3.7.3 Long Short Term Memory (LSTM)

Long Short-Term Memory (LSTM) is an enhanced version of the Recurrent Neural Networks (RNNs) designed by Hochreiter and Schmidhuber. LSTMs can capture long-term dependencies in sequential data making them ideal for tasks like language translation, speech recognition and time series forecasting. Unlike traditional RNNs which use a single hidden state passed through time LSTMs introduce a memory cell that holds information over extended periods addressing the challenge of learning long-term dependencies.

3.7.3.1 Structure of LSTM

LSTM structure involves the memory cell which is controlled by three gates:

1. Input gate: Controls what information is added to the memory cell.
2. Forget gate: Determines what information is removed from the memory cell.
3. Output gate: Controls what information is output from the memory cell.

This allows LSTM networks to selectively retain or discard information as it flows through the network which allows them to learn long-term dependencies. The network has a hidden state which is like its short-term memory. This memory is updated using the current input, the previous hidden state and the current state of the memory cell.

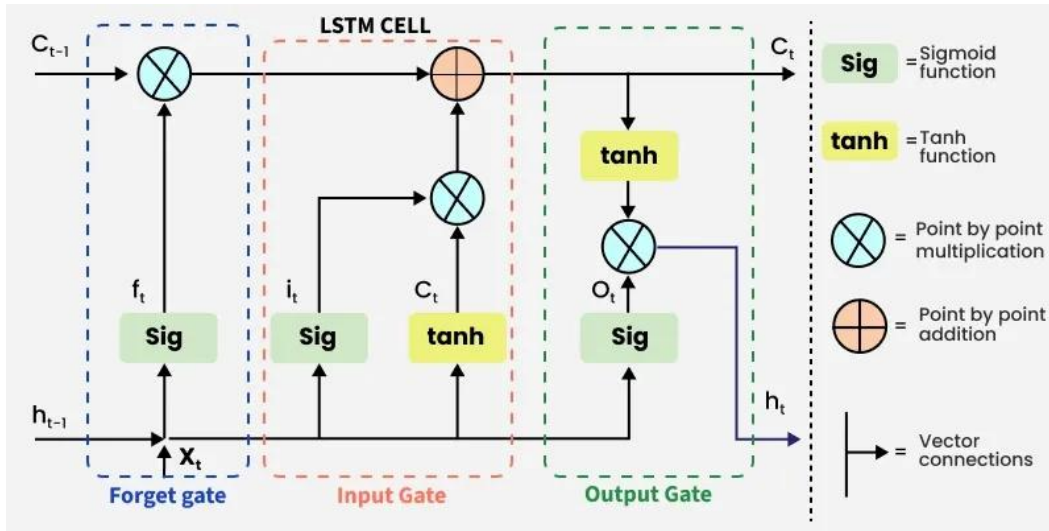


Figure 3.11: Structure of LSTM

1. Forget Gate

The information that is no longer useful in the cell state is removed with the forget gate. Two inputs x_t (input at the particular time) and h_{t-1} (previous cell output) are fed to the gate and multiplied with weight matrices followed by the addition of bias. The resultant is passed through sigmoid activation function which gives output in range of $[0,1]$. If for a particular cell state the output is 0 or near to 0, the piece of information is forgotten and for output of 1 or near to 1, the information is retained for future use. The equation for the forget gate is:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Where:

- W_f represents the weight matrix associated with the forget gate.
- $[h_{t-1}, x_t]$ denotes the concatenation of the current input and the previous hidden state.
- b_f is the bias with the forget gate.
- σ is the sigmoid activation function.

2. Input Gate

The addition of useful information to the cell state is done by the input gate. First the information is regulated using the sigmoid function and filter the values to be remembered similar to the forget gate using inputs h_{t-1} and x_t . Then, a vector is created using $\tanh$ function that gives an output from -1 to +1 which contains all the possible values from h_{t-1} and x_t . At last the values of the vector and the regulated values are multiplied to obtain the useful information. The equation for the input gate is:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$C_t^\wedge = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

We multiply the previous state by f_t effectively filtering out the information we had decided to ignore earlier. Then we add $i_t \odot C_t$ which represents the new candidate values scaled by how much we decided to update each state value.

$$C_t = f_t \odot C_{t-1} + i_t \odot C_t^\wedge$$

Where

- $\odot$ denotes element-wise multiplication
- $\tanh$ is activation function

3. Output Gate

The output gate is responsible for deciding what part of the current cell state should be sent as the hidden state (output) for this time step. First, the gate uses a sigmoid function to determine which information from the current cell state will be output. This is done using the previous hidden state h_{t-1} and the current input x_t :

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

Next, the current cell state C_t is passed through a $\tanh$ activation to scale its values between -1 and $+1$. Finally, this transformed cell state is multiplied element-wise with o_t to produce the hidden state h_t :

$$h_t = o_t \odot \tanh(C_t)$$

Here:

- o_t is the output gate activation.
- C_t is the current cell state.
- $\odot$ represents element-wise multiplication.

- σ is the sigmoid activation function.

This hidden state h_t is then passed to the next time step and can also be used for generating the output of the network.

Due to their ability to capture long-term dependencies, LSTM models have been widely applied in financial time series prediction.

3.7.4 One-Dimensional Convolutional Neural Network (1D CNN)

A One-Dimensional Convolutional Neural Network (1D CNN) is a deep learning model designed to process one-dimensional sequential data such as time-series signals, textual data, or sensor readings.

Unlike traditional neural networks, 1D CNN uses convolution operations along a single dimension to automatically extract meaningful patterns and features from sequential inputs.

The model applies filters (kernels) that slide across the input sequence to identify important local patterns such as trends, spikes, or periodic behaviour. These extracted features are then used for tasks such as prediction, classification, or pattern recognition.

1D CNN is widely used in applications where data is arranged in ordered sequences, such as stock prices, speech signals, and sensor data.

3.7.4.1 Difference Between 1D CNN and 2D CNN

The main difference between 1D Convolutional Neural Network (1D CNN) and 2D Convolutional Neural Network (2D CNN) lies in the type of data they process and the direction in which the convolution operation is applied.

A 1D CNN is designed to work with one-dimensional sequential data, where the data is arranged in a single sequence over time. Examples include time-series data, stock prices, audio signals, and text sequences. In this model, the convolution filter moves along one direction (the sequence or time axis) to capture patterns and trends in the data.

In contrast, a 2D CNN is designed to process two-dimensional data, mainly images. The input is represented as a matrix of pixels with height and width. In this case, the convolution filter

moves in two directions (both horizontally and vertically) to extract spatial features such as edges, textures, and shapes.

Another difference is that 1D CNN focuses on learning temporal patterns in sequential data, while 2D CNN focuses on learning spatial patterns in images. Because 1D CNN operates on a single dimension, it generally requires less computational power and fewer parameters compared to 2D CNN.

In summary, 1D CNN is mainly used for sequential data analysis such as time-series forecasting and signal processing, whereas 2D CNN is primarily used for image-related tasks such as image classification and object detection.

3.7.4.2 Structure of 1D Convolutional Neural Network (1D CNN)

The structure of a 1D Convolutional Neural Network (1D CNN) consists of several layers that work together to extract patterns from sequential data and make predictions. The main components of the 1D CNN architecture are explained below.

1. Input Layer

The input layer receives the one-dimensional sequential data. The data is usually arranged as a sequence of values over time. For example, in time-series forecasting or stock price prediction, the input may be a sequence of past stock prices. The input layer passes this sequence to the convolution layer for further processing.

2. Convolution Layer

The convolution layer is responsible for extracting important features from the input sequence. In this layer, filters (also called kernels) slide along the input data and perform convolution operations. These filters detect local patterns such as trends, spikes, or repeating patterns within the sequence.

The convolution operation can be mathematically expressed as:

$$y_t = \sum_{i=1}^k x_{t+i} \cdot w_i$$

where:

- x = input sequence
- w = filter weights
- k = kernel size
- y_t = resulting feature value.

3. Activation Layer

After the convolution operation, an activation function is applied to introduce non-linearity into the model. The most commonly used activation function is ReLU (Rectified Linear Unit). This layer helps the network learn complex patterns from the extracted features.

$$f(x)=\max(0,x)$$

4. Pooling Layer

The pooling layer reduces the size of the feature maps generated by the convolution layer. This helps decrease computational complexity and prevents overfitting. The most commonly used pooling method is Max Pooling, which selects the maximum value from each region of the feature map.

5. Flatten Layer

The flatten layer converts the pooled feature maps into a one-dimensional vector. This step prepares the data to be processed by the fully connected layer.

6. Fully Connected (Dense) Layer

The fully connected layer connects every neuron from the previous layer to each neuron in the current layer. This layer learns the relationships between the extracted features and the target output.

7. Output Layer

The output layer produces the final prediction of the model. In regression tasks such as stock price prediction, the output layer provides a numerical value representing the predicted future value.

3.7.4.3 Applications of 1D CNN

- Time Series Forecasting
 - Stock price prediction
 - Weather forecasting
- Speech Recognition
 - Voice signal analysis
- Text Processing
 - Sentiment analysis
 - Document classification
- Sensor Data Analysis
 - IoT signals
 - Activity recognition
- Biomedical Signal Processing
 - ECG signal analysis
 - EEG signal classification

3.7.5 CNN-LSTM

A CNN–LSTM hybrid model is a deep learning architecture that combines the strengths of Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks to process sequential data.

In this hybrid approach:

- CNN layers are used to extract important features and local patterns from the input data.
- LSTM layers are used to capture long-term temporal dependencies in the sequence.

This combination allows the model to automatically learn both spatial features and temporal relationships, making it highly effective for time-series forecasting tasks.

3.7.5.1 Working of CNN–LSTM Model

1. Input Sequence

The model first receives sequential input data arranged in time order. For example, in stock price prediction, the input may consist of a sequence of historical stock prices over several time steps.

2. Feature Extraction using CNN

The input sequence is passed through 1D convolution layers. In this stage, filters (kernels) slide across the sequence and extract important local patterns, such as short-term trends or fluctuations in the data.

The convolution operation helps the model automatically identify useful features from the raw input data.

3. Activation and Pooling

After convolution, an activation function such as ReLU is applied to introduce non-linearity. Then, a pooling layer (usually max pooling) is used to reduce the dimensionality of the feature maps while retaining the most significant information. This step improves computational efficiency and reduces overfitting.

4. Feature Sequence Formation

The extracted features from the CNN layers are reshaped into a sequence format suitable for the LSTM layer. These features represent important patterns detected from the input data.

5. Temporal Learning using LSTM

The processed feature sequence is then passed to the LSTM layer. The LSTM network analyses the sequential dependencies between the extracted features and learns long-term relationships in the data using its memory cells and gating mechanisms. This step enables the model to understand how past patterns influence future outcomes.

6. Fully Connected Layer

The output from the LSTM layer is passed to a fully connected (dense) layer, which combines the learned features and prepares them for final prediction.

7. Output Prediction

Finally, the output layer generates the predicted result. In time-series forecasting tasks, the output represents the predicted future value, such as the next stock price. This hybrid approach allows the model to capture both local patterns and long-term dependencies in sequential data, resulting in improved prediction performance for time-series problems.

3.8 Model Evaluation Metrics

To evaluate the forecasting performance of the deep learning models, several statistical error metrics were used. Model evaluation metrics help measure how accurately the predicted values match the actual stock prices. In this study, four commonly used performance metrics were employed: Mean Absolute Error (MAE), Root Mean Square Error (RMSE), Mean Absolute Percentage Error (MAPE), and prediction accuracy.

These metrics provide quantitative measures of prediction error and allow comparison of the performance of different models used in the study.

3.8.1 Mean Absolute Error (MAE)

Mean Absolute Error (MAE) measures the average absolute difference between the predicted values and the actual observed values. It provides an indication of how close the model predictions are to the actual data.

The MAE is calculated as the average of the absolute differences between predicted and actual values.

The formula for MAE is given as:

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

where:

- y_i represents the actual value
- $\hat{y}_i$ represents the predicted value
- n represents the total number of observations

3.8.2 Root Mean Square Error (RMSE)

Root Mean Square Error (RMSE) measures the square root of the average squared differences between predicted values and actual values. RMSE penalizes larger errors more heavily than MAE, making it sensitive to large prediction errors.

The RMSE is calculated using the following formula:

$$RMSE = \sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$$

where:

- y_i represents the actual value
- $\hat{y}_i$ represents the predicted value
- n represents the number of observations

3.8.3 Mean Absolute Percentage Error (MAPE)

Mean Absolute Percentage Error (MAPE) measures the average percentage difference between the predicted values and the actual values. It provides an easily interpretable measure of forecasting accuracy expressed as a percentage.

The formula for MAPE is given as:

$$MAPE = \left(\frac{100}{n} \right) \sum \left| \frac{(y_i - \hat{y}_i)}{y_i} \right|$$

where:

- y_i represents the actual value
- $\hat{y}_i$ represents the predicted value
- n represents the number of observations

3.8.4 Prediction Accuracy

Prediction accuracy represents the overall performance of the forecasting model in predicting stock prices. In this study, accuracy was derived from the MAPE value using the following relationship:

$$Accuracy = 100 - MAPE$$

This measure provides a simple way to express the prediction performance of the models as a percentage.

3.9 Methodology

1. Data Collection

- Historical stock price data of Infosys Limited was collected from Yahoo Finance.
- The dataset contains weekly observations of stock prices.
- The variables included in the dataset are Open, High, Low, Close, and Volume.

2. Feature Engineering

- Candlestick pattern analysis was used to create additional variables using the Open, High, Low, and Close prices.
- These variables were tested to examine whether they improve model performance.
- The results showed that candlestick variables did not significantly improve prediction accuracy, so the final dataset used only the original stock price variables.

3. Data Preprocessing

Several preprocessing steps were performed to prepare the dataset for modeling.

- Data cleaning and checking for missing values
- Converting the Date column into datetime format
- Sorting the dataset in chronological order
- Scaling the data using Min-Max normalization

4. Time Series Sequence Creation

- The dataset was converted into sequences using the sliding window method.
- Past observations were used as input to predict the next closing price of the stock.

5. Train–Test Split

- The dataset was divided into training data and testing data.
- Earlier observations were used for training the models.
- Later observations were used to test the prediction performance.

6. Walk-Forward Validation

- Walk-forward validation was applied to evaluate the forecasting models.
- The model predicts the next value and then moves forward step by step across the test data.

7. Model Development

Five deep learning models were developed and compared in this study.

- Recurrent Neural Network (RNN)
- Gated Recurrent Unit (GRU)
- Long Short-Term Memory (LSTM)
- Convolutional Neural Network (CNN)
- CNN-LSTM Hybrid Model

8. Model Evaluation

The models were evaluated using the following performance metrics:

- MAE (Mean Absolute Error)
- RMSE (Root Mean Square Error)
- MAPE (Mean Absolute Percentage Error)
- Prediction Accuracy

9. Stock Price Forecasting

- After evaluating all models, the best-performing model was selected.
- This model was used to forecast the next week's stock price.
- The forecast results include:
 - Predicted stock price
 - Predicted return
 - Predicted return percentage

The forecasting results are presented and discussed in the results and analysis chapter.

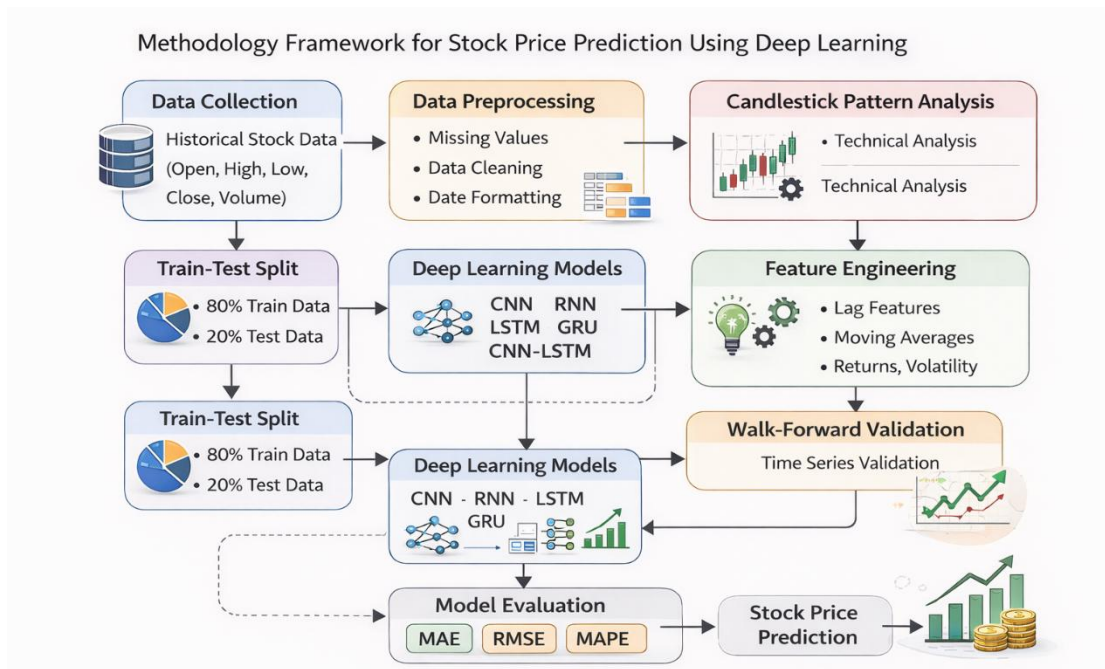


Figure 3.12: Project Flowchart

3.10 Tools and Technologies Used

- **Python** was used for data analysis and model implementation.
- **Pandas** – data manipulation and preprocessing
- **NumPy** – numerical computations
- **Matplotlib** – data visualization
- **Seaborn** – statistical visualization
- **TensorFlow / Keras** – deep learning model development
- **Scikit-learn** – preprocessing and evaluation metrics

These tools were used to perform data analysis, model development, and forecasting efficiently.

CHAPTER – 4

RESULTS AND DISCUSSIONS

This chapter presents the results obtained from the analysis of the dataset and the forecasting models developed in this study. The purpose of this chapter is to understand the behavior of the stock price data and evaluate the performance of the deep learning models used for forecasting.

First, descriptive statistics are presented to summarize the main characteristics of the dataset. After that, different plots such as the closing price trend, stock returns, volatility, lag relationship, and rolling statistics are analysed to understand the patterns in the stock price data. Finally, the results of the deep learning models are presented and compared to identify the model that provides the best forecasting performance.

4.1 Descriptive Statistics

Descriptive statistics help summarize the main characteristics of the dataset and provide a basic understanding of the stock price variables used in this study.

Table 4.1 presents the descriptive statistics for the variables Open, High, Low, Close, and Volume in the dataset.

Statistic	Close	High	Low	Open	Volume
Count	784	784	784	784	784
Mean	773.36	792.14	753.86	771.77	40,101,510
Standard Deviation	528.10	540.37	515.68	527.20	22,085,660
Minimum	191.27	196.91	183.43	192.28	6,959,880
25th Percentile	355.97	363.20	348.68	356.40	27,780,430
Median (50%)	537.91	560.14	509.07	534.16	35,105,970
75th Percentile	1340.21	1376.72	1312.29	1336.84	44,922,980
Maximum	1942.22	1948.78	1856.31	1938.09	243,314,500

Table 4.1: Descriptive Statistics of Stock Price Variables

- The dataset contains 784 observations for all variables, which indicates that the dataset is complete and does not contain missing values.
- The average closing price of the stock is approximately 773.36, while the average opening price is around 771.77. This shows that the opening and closing prices are generally close to each other.
- The standard deviation values are relatively high for the price variables, which indicates that the stock price has experienced noticeable fluctuations over the study period.

- The minimum closing price is about 191.27, while the maximum closing price is about 1942.22. This large difference shows that the stock price has increased significantly over time.
- The median closing price is approximately 537.91, which means that half of the observations have closing prices below this value and the other half have values above it.
- The 75th percentile value (1340.21) indicates that about 25% of the stock prices are above this level, which reflects the higher price levels observed in recent years.
- The average trading volume is approximately 40 million shares, indicating the typical level of trading activity.
- The maximum trading volume exceeds 243 million shares, suggesting that there were certain periods with very high trading activity in the market.
- Overall, the descriptive statistics indicate that the stock price has shown considerable growth over time while also experiencing fluctuations, and trading volume varies depending on market activity.

4.2 Closing Price Trend Analysis

The closing price trend analysis helps understand how the stock price has changed over time. By analysing the trend of the closing price, it is possible to observe the long-term movement and growth pattern of the stock.



Figure 4.1: Closing Price Trend of Infosys Stock

The plot shows the movement of the closing price from 2011 to 2026. The stock price shows a clear upward trend over the years, indicating long-term growth in the value of the Infosys stock. In the early years (around 2011–2014), the stock price remained relatively low and stable.

From 2015 onwards, the stock price started increasing gradually. A significant rise in the stock price can be observed after 2020, showing strong growth during that period. Although the general trend is upward, the plot also shows short-term fluctuations, which are common in stock markets due to market conditions, investor behaviour, and economic factors. Overall, the trend indicates that the stock has experienced long-term growth with periodic fluctuations, making it suitable for time series forecasting using deep learning models.

4.3 Stock Return Analysis

Stock returns represent the percentage change in the stock price from one period to the next. Analysing returns helps understand how the stock price moves over time and how frequently the price increases or decreases. Return analysis is useful for identifying fluctuations and variability in stock price movements.

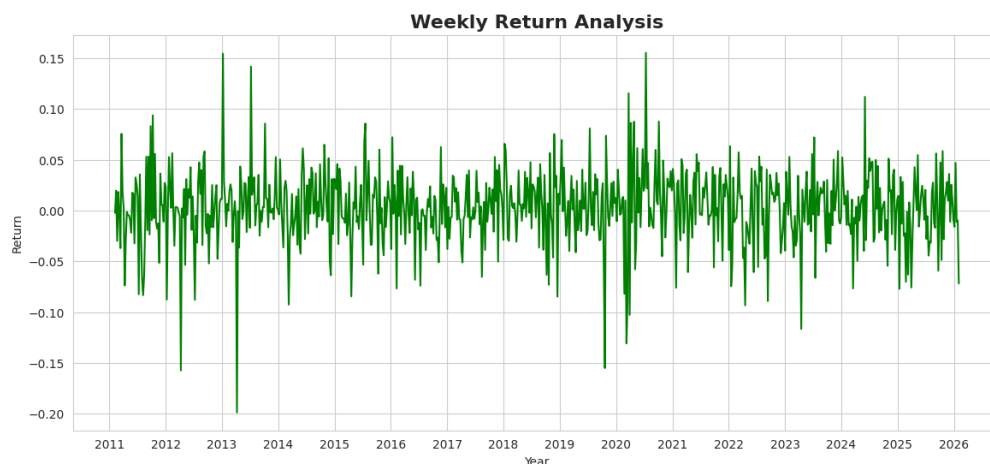


Figure 4.2: Weekly Return Plot

The plot shows the weekly returns of the Infosys stock price over the study period. Most of the return values are close to zero, which means the weekly price changes are usually small. The returns fluctuate between positive and negative values, indicating that the stock price sometimes increases and sometimes decreases from week to week. Some large spikes can be observed in the plot, which represent weeks where the stock experienced large price changes. Overall, the returns move around zero with occasional large changes, which shows that stock prices experience short-term fluctuations over time.

4.4 Lag Plot Analysis

Lag analysis helps understand the relationship between the current stock price and its previous values. It is used to examine whether past price movements influence future price behaviour.

If a clear pattern appears in the lag plot, it indicates that the time series has a strong dependency on its previous values.

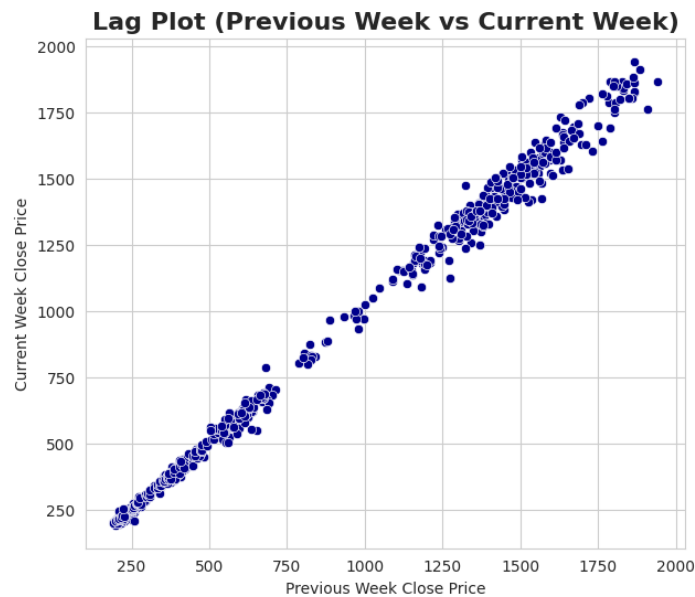


Figure 4.3: Lag Plot of Closing Price

The lag plot shows a clear positive relationship between the current closing price and the previous week's closing price.

Most of the points are clustered along a diagonal pattern, indicating that the current price is strongly influenced by the past price.

This pattern suggests that the stock price series has autocorrelation, meaning past values help explain future values.

The presence of this relationship confirms that the dataset contains time-dependent patterns, making it suitable for time series forecasting using deep learning models.

4.5 Rolling Statistics Analysis

Rolling statistics are used to analyse how the statistical properties of a time series change over time.

In this study, rolling mean and rolling standard deviation were calculated to examine the trend and variability of the stock price.

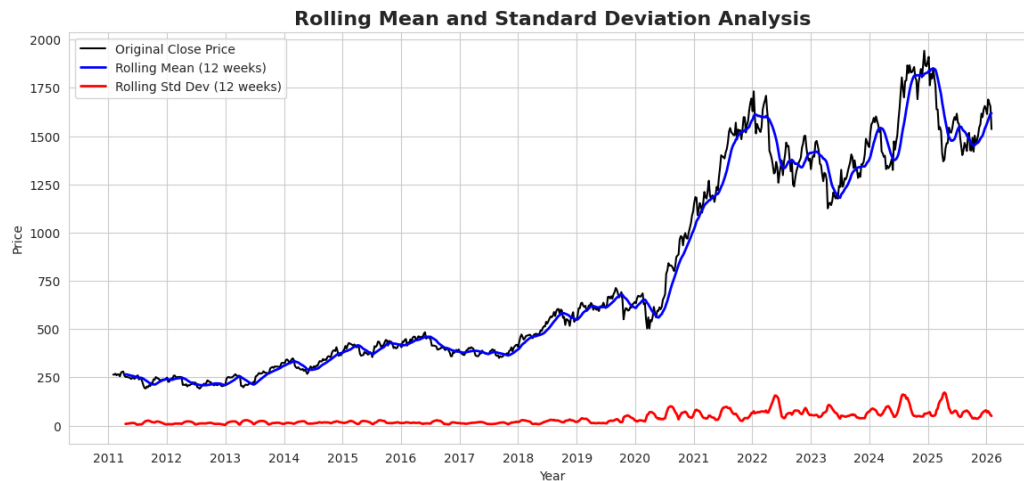


Figure 4.4: Rolling Mean and Rolling Standard Deviation

The rolling mean represents the moving average of the stock price and helps smooth short-term fluctuations to reveal the underlying trend. The rolling standard deviation measures the variability of the stock price over time. From the plot, the rolling mean follows the overall upward trend of the stock price, indicating long-term growth. The rolling standard deviation shows variations in volatility during different periods, with some sections displaying higher fluctuations. These observations indicate that the stock price series exhibits both trend and changing variability over time.

4.6 Model results

In this study, several deep learning models were developed to forecast the future stock price of Infosys using historical stock price data. The models used include RNN, GRU, LSTM, CNN, and CNN-LSTM. The performance of these models was evaluated using common forecasting metrics such as Mean Absolute Error (MAE), Root Mean Square Error (RMSE), Mean Absolute Percentage Error (MAPE), and prediction accuracy. It is important to note that the MAE and RMSE values appear relatively large. This is mainly because the stock price values themselves are large, ranging from around ₹191 to nearly ₹1942 in the dataset. Since MAE and RMSE measure the error in the actual price units, their values tend to be higher when the scale of the data is large. Therefore, these values should be interpreted relative to the price range of the dataset rather than as absolute numbers.

To better evaluate model performance, MAPE and accuracy are also considered, since they measure the prediction error in percentage terms. In this study, the MAPE values are relatively low and the accuracy values are above 97%, which indicates that the models provide reliable forecasts despite the higher MAE and RMSE values. Therefore, the observed error values are

acceptable for stock price forecasting, and the models are still able to capture the overall trend and patterns in the data effectively.

4.6.1 Recurrent Neural Network (RNN)

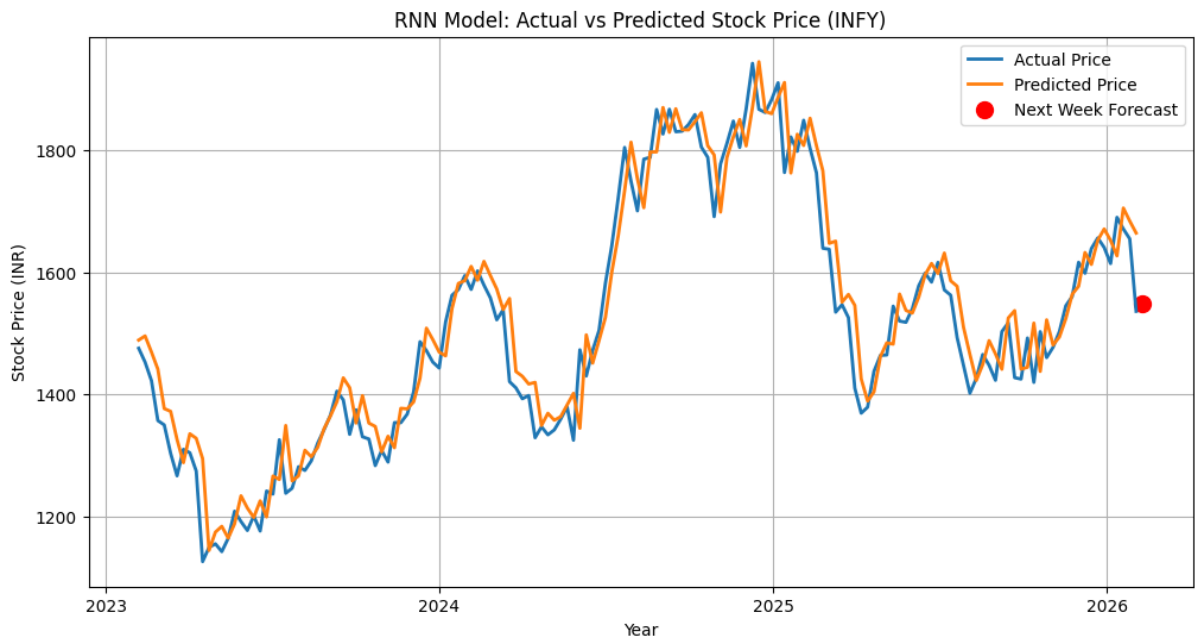


Figure 4.5: Actual vs Predicted Stock Price using the RNN Model

The figure shows the comparison between the actual closing prices and the predicted prices generated by the RNN model. It can be observed that the predicted values follow the overall movement of the actual stock prices. This indicates that the RNN model is able to capture the general trend and patterns in the stock price data. However, small differences between the predicted and actual values can still be seen in some periods. The RNN model was also used to forecast the stock price for the next week based on the latest available data.

Current Price	1535.80
Predicted Price	1547.97
Predicted Return	0.79%
MAE	40.9568
RMSE	52.8356
MAPE	2.75%
Accuracy	97.25%

Table 4.2: Next week price and evaluation metrics of RNN model

The forecast result suggests a slight increase in the stock price for the upcoming week. The RNN model is able to capture the overall trend of the stock price, as the predicted values closely

follow the actual price movements in the plot. Although the model follows the general pattern of the stock price, small differences between the predicted and actual values can be observed in some periods. The MAE and RMSE values indicate the average prediction error in price units, which is reasonable considering the wide range of stock prices in the dataset. The MAPE value of 2.75% shows that the prediction error is very small in percentage terms, indicating good forecasting performance. The accuracy of 97.25% suggests that the model predicts the stock price with high reliability. Overall, the RNN model performs well in capturing the temporal patterns in the stock price data and provides reliable short-term forecasting results.

4.6.2 Gated Recurrent Unit (GRU)

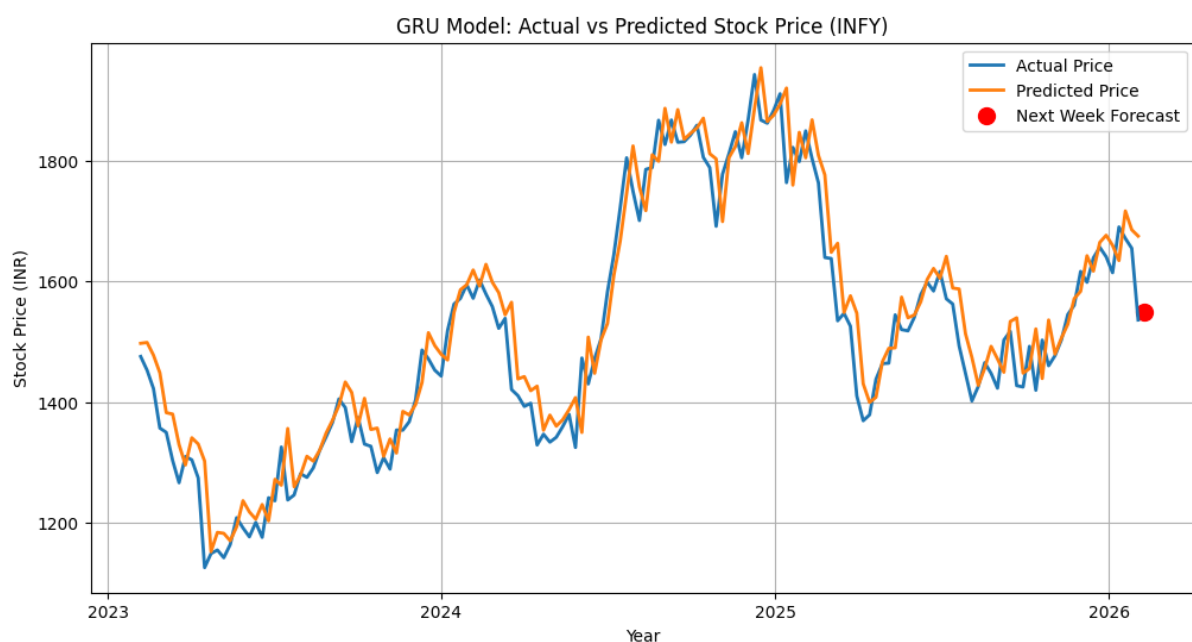


Figure 4.6: Actual vs Predicted Stock Price using the GRU Model

The figure compares the actual closing prices with the predicted prices generated by the GRU model. The predicted values follow the overall trend of the actual stock prices, indicating that the model successfully captures the temporal patterns in the dataset. Although small deviations between the predicted and actual values can be observed in some periods, the general movement of the stock price is accurately reflected by the model.

The GRU model was also used to forecast the stock price for the next week based on the most recent data.

Current Price	1535.80
Predicted Price	1549.79
Predicted Return	0.91%
MAE	43.1909
RMSE	55.8884
MAPE	2.91%
Accuracy	97.09%

Table 4.3: Next week price and evaluation metrics of GRU model

The forecast result indicates a slight increase in the stock price for the next week. The GRU model successfully captures the trend and movement of the stock price over time. The predicted values closely follow the actual price pattern, indicating good forecasting capability. The MAPE value of 2.91% shows that the percentage prediction error is very low, which indicates good prediction accuracy. The accuracy of 97.09% suggests that the model provides reliable forecasts for the stock price. Overall, the GRU model demonstrates strong performance in learning time series patterns and producing accurate stock price predictions.

4.6.3 Long Short-Term Memory (LSTM)

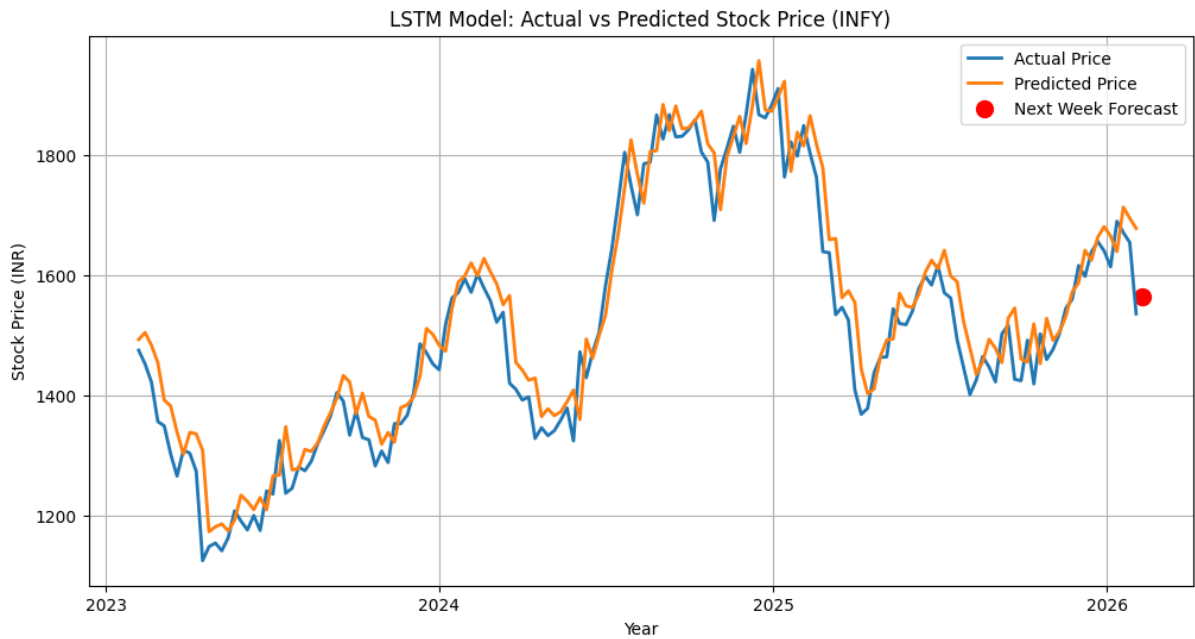


Figure 4.7: Actual vs Predicted Stock Price using the LSTM Model

The figure compares the actual closing prices with the predicted prices generated by the LSTM model. The predicted values closely follow the actual stock price movements, indicating that the model is able to capture the overall trend and fluctuations in the data. Although slight differences between the predicted and actual values can be observed in certain periods, the

model successfully tracks the general movement of the stock price. The LSTM model was also used to forecast the next week's stock price based on the most recent data.

Current Price	1535.80
Predicted Price	1563.44
Predicted Return	1.80%
MAE	44.5015
RMSE	56.7237
MAPE	3.01%
Accuracy	96.99%

Table 4.4: Next week price and evaluation metrics of LSTM model

The forecast result suggests a moderate increase in the stock price for the upcoming week. The LSTM model successfully captures the overall trend and fluctuations of the stock price. The predicted values closely follow the actual price movements, indicating good forecasting capability. The MAPE value of 3.01% indicates a small prediction error, showing that the model performs well in forecasting stock prices. The accuracy of 96.99% suggests that the model provides reliable predictions for the stock price. Overall, the LSTM model demonstrates strong performance in learning long-term dependencies in the stock price data and producing accurate forecasts.

4.6.4 Convolutional Neural Network (CNN)

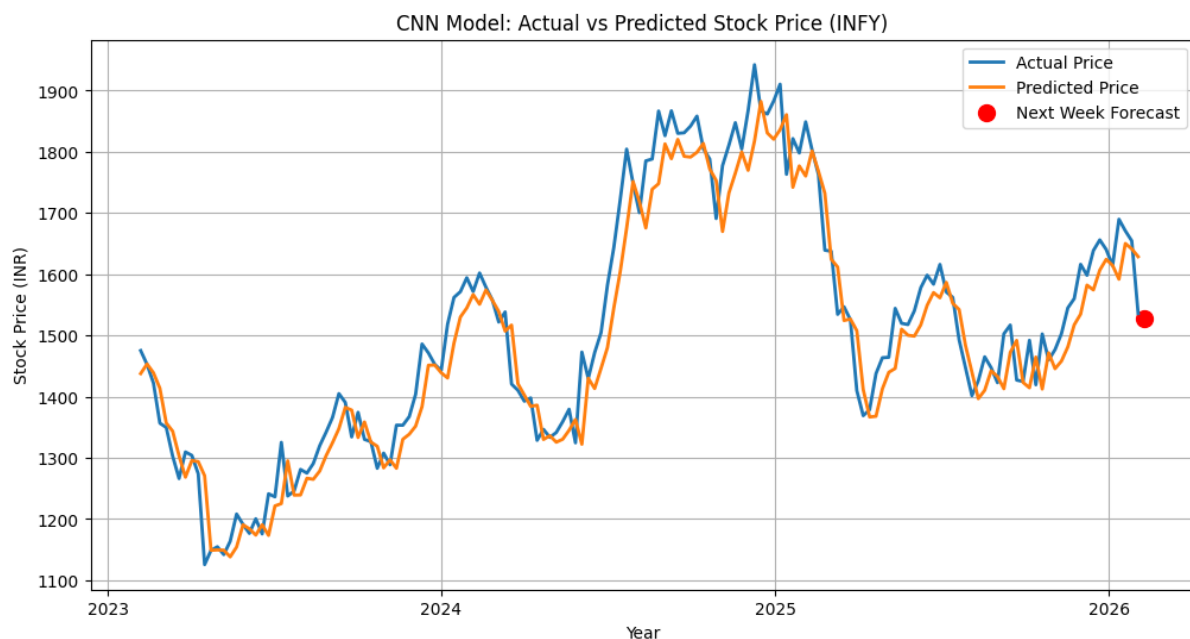


Figure 4.8: Actual vs Predicted Stock Price using the CNN Model

The figure compares the actual closing prices with the predicted prices generated by the CNN model. The predicted values closely follow the actual price movements, indicating that the model successfully captures the short-term fluctuations and general trend of the stock price. Some small deviations between predicted and actual values can be observed, but the model overall tracks the price movement effectively. The CNN model was also used to forecast the next week's stock price based on the latest available data.

Current Price	1535.80
Predicted Price	1528.34
Predicted Return	-0.49%
MAE	42.7647
RMSE	55.1892
MAPE	2.80%
Accuracy	97.20%

Table 4.5: Next week price and evaluation metrics of CNN model

The forecast result indicates a slight decrease in the stock price for the upcoming week. The CNN model successfully captures the short-term patterns and fluctuations in the stock price data. The predicted values follow the actual stock price trend closely, indicating good forecasting performance. The MAPE value of 2.80% shows that the prediction error is relatively low, indicating accurate predictions. The accuracy of 97.20% suggests that the model performs well in forecasting the stock price. Overall, the CNN model demonstrates strong ability in learning patterns from the time series data and producing reliable predictions.

4.6.5 CNN-LSTM Hybrid Model

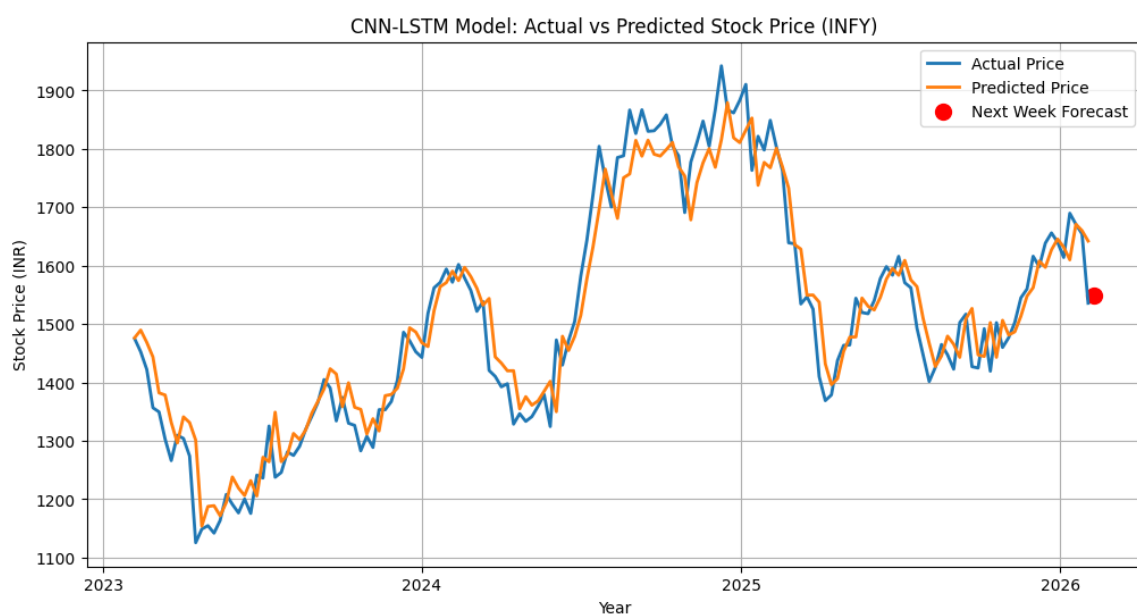


Figure 4.9: Actual vs Predicted Stock Price using the CNN-LSTM Model

The figure shows the comparison between the actual closing prices and the predicted prices generated by the CNN–LSTM model. It can be observed that the predicted values follow the actual stock price movements closely. The model successfully captures both the trend and short-term fluctuations in the stock price data. Although small deviations between predicted and actual values occur in some periods, the model overall tracks the stock price pattern effectively. The CNN–LSTM model was also used to forecast the next week’s stock price.

Current Price	1535.80
Predicted Price	1548.47
Predicted Return	0.83%
MAE	41.8320
RMSE	53.8368
MAPE	2.80%
Accuracy	97.20%

Table 4.5: Next week price and evaluation metrics of CNN-LSTM model

The forecast result indicates a slight increase in the stock price for the upcoming week. The CNN–LSTM model effectively captures both short-term patterns and long-term dependencies in the stock price data. The predicted values follow the actual stock price trend closely, indicating good forecasting capability. The MAPE value of 2.80% indicates a low percentage prediction error, showing strong prediction performance. The accuracy of 97.20% suggests that the model provides reliable forecasts for the stock price.

Overall, the hybrid CNN–LSTM model demonstrates strong predictive performance by combining the advantages of CNN and LSTM architectures.

4.7 Model Comparison

In this study, multiple deep learning models were developed to forecast the stock price of Infosys. These models include RNN, GRU, LSTM, CNN, and CNN–LSTM. To identify the most effective model, the performance of each model was compared using different evaluation metrics.

The models were evaluated using Mean Absolute Error (MAE), Root Mean Square Error (RMSE), Mean Absolute Percentage Error (MAPE), and prediction accuracy. Lower values of MAE, RMSE, and MAPE indicate better prediction performance, while higher accuracy indicates a more reliable forecasting model.

Model	MAE	RMSE	MAPE	Accuracy
RNN	40.9568	52.8356	2.75%	97.25%
GRU	43.1909	55.8884	2.91%	97.09%
LSTM	44.5015	56.7237	3.01%	96.99%
CNN	42.7647	55.1892	2.80%	97.20%
CNN-LSTM	41.8320	53.8368	2.80%	97.20%

Table 4.6: Comparison of Deep Learning Model Performance

Among all models, the RNN model achieved the lowest MAE and RMSE values, indicating the smallest prediction errors. The RNN model also has the lowest MAPE value (2.75%), which indicates the smallest percentage prediction error. The CNN-LSTM and CNN models also perform well, with relatively low error values and high accuracy. The LSTM model shows slightly higher error values compared to the other models, although it still provides reliable predictions.

Overall, all models achieve accuracy values above 96%, indicating strong forecasting performance. Based on the comparison results, the RNN model performs slightly better than the other models in terms of prediction accuracy and error metrics.

To visualize the comparison more clearly, a bar chart of model accuracies is presented in Figure 4.10.

Model	Current Price	Predicted Price	Predicted Return (%)
RNN	1535.80	1547.97	0.79%
GRU	1535.80	1549.79	0.91%
LSTM	1535.80	1563.44	1.80%
CNN	1535.80	1528.34	-0.49%
CNN-LSTM	1535.80	1548.47	0.83%

Table 4.7: Next Week Forecast Results

Most models predict a slight increase in the stock price for the next week. The LSTM model predicts the highest increase in stock price. The CNN model predicts a small decline, indicating possible short-term fluctuation. The predictions from different models are relatively close, suggesting stable forecasting behaviour. Overall, the results indicate a moderate upward expectation for the stock price.

4.8 Summary of Results

This chapter presented the results obtained from the analysis of the dataset and the forecasting models developed in this study. Descriptive statistics and various analytical plots were used to understand the behaviour of the stock price data.

Five deep learning models were implemented to forecast the stock price, including RNN, GRU, LSTM, CNN, and CNN-LSTM. The performance of these models was evaluated using MAE, RMSE, MAPE, and accuracy metrics. The comparison results showed that the RNN model achieved the best performance among the models used in this study.

The forecasting results indicate that the stock price is expected to experience a slight increase in the upcoming week, although minor fluctuations may occur.

CHAPTER – 5

CONCLUSION

5.1 Conclusion

This study aimed to forecast the stock price of Infosys Limited using deep learning models. Historical stock price data consisting of Open, High, Low, Close, and Volume variables were collected and analysed to understand the behaviour of the stock price over time. Several analytical techniques were applied to examine the characteristics of the dataset, including trend analysis, return analysis, volatility analysis, lag relationships, and rolling statistics. These analyses helped identify patterns and fluctuations present in the stock price data.

To forecast future stock prices, five deep learning models were developed and evaluated, namely Recurrent Neural Network (RNN), Gated Recurrent Unit (GRU), Long Short-Term Memory (LSTM), Convolutional Neural Network (CNN), and a hybrid CNN-LSTM model. These models were trained using historical time series data and evaluated using performance metrics such as Mean Absolute Error (MAE), Root Mean Square Error (RMSE), Mean Absolute Percentage Error (MAPE), and prediction accuracy.

The comparison of model performance showed that all models performed well in predicting the stock price, with accuracy values above 96%. Among the models used in this study, the RNN model demonstrated the best overall performance, achieving the lowest error values and the highest prediction accuracy. The results also indicate that deep learning models are effective in capturing temporal patterns and fluctuations in stock price data.

The forecasting results suggest a slight increase in the stock price for the upcoming week, although minor fluctuations may occur due to market conditions. Overall, the findings of this study show that deep learning techniques can provide reliable predictions for stock price forecasting and can be useful tools for financial analysis and decision-making.

The study found some important points. First, the stock price of Infosys shows an overall increasing trend with some short-term ups and downs. Second, the return and volatility analysis showed that the stock price changes regularly over time.

Another finding is that past stock prices affect future prices, which means the data is suitable for time series forecasting. All the models used in the study gave high accuracy above 96%. Among them, the RNN model gave the best results. The prediction also shows that the stock price may increase slightly in the coming week.

Even though the study gave good results, it has some limitations. The study used only past stock price data and did not include other important factors like economic conditions, company performance, or market news. These factors can also affect stock prices.

Also, the study used weekly data, so it may not capture daily or short-term changes properly. Another limitation is that only a few deep learning models were used, and other advanced models were not tested. Because of these reasons, there is still scope to improve the study.

In the future, this study can be improved in many ways. More data like technical indicators, economic factors, and market sentiment can be added to improve accuracy. Advanced models like Transformer and attention-based models can also be used.

The study can also be extended to more companies or different sectors to compare results. Using a combination of models (ensemble methods) may also improve prediction accuracy. In addition, using daily data instead of weekly data can help in capturing short-term changes better. Overall, future work can make the model more accurate and useful in real-world situations.

5.2 References

- [1] H. Bao, X. Yue, and Y. Rao, “A comprehensive review of data-driven neural networks for stock market forecasting,” *Information Fusion*, vol. 103, 2025.
 - [2] S. Karunamurthy, R. Kumar, and P. Singh, “Stock price prediction using deep learning and Monte Carlo simulation,” 2024, available: ResearchGate.
 - [3] A. Nugroho, B. Pratama, and R. Wibowo, “A hybrid ensemble framework integrating ARIMA, random forest, and LSTM for stock market forecasting,” *Procedia Computer Science*, vol. 225, 2025.
 - [4] E. Giantsidi, “Deep learning-based financial forecasting: A systematic review of recent advances,” *Applied Soft Computing*, vol. 157, 2025.
 - [5] A. Barua, S. Chatterjee, and M. Roy, “Comparative analysis of deep learning models for stock price prediction in the Indian market,” *AI*, vol. 3, no. 4, pp. 1–18, 2024.
 - [6] Y. Li, J. Wang, and K. Zhang, “Stock price forecasting using LSTM neural networks: A study on technology sector stocks,” *Computation*, vol. 13, no. 1, 2024.
 - [7] B. Dinda, “Temporal gated recurrent neural networks with Bayesian optimization for stock index forecasting,” *arXiv preprint arXiv:2406.02604*, 2024.
 - [8] S. Gülmez, “A hybrid LSTM–Seahorse Optimization algorithm for stock price prediction,” *International Journal of System Assurance Engineering and Management*, 2025.
 - [9] M. Szydłowski and P. Chudziak, “Hidformer: Transformer-based architecture for stock price forecasting,” *arXiv preprint arXiv:2412.19932*, 2024.
 - [10] A. Radfar, M. Ahmadi, and H. Rahimi, “Deep learning approaches for financial trend prediction and technical analysis,” *Humanities and Social Sciences Communications*, vol. 12, 2025.
- <https://www.geeksforgeeks.org/machine-learning/introduction-to-recurrent-neural-network/>
- <https://www.geeksforgeeks.org/machine-learning/gated-recurrent-unit-networks/>
- <https://www.geeksforgeeks.org/deep-learning/deep-learning-introduction-to-long-short-term-memory/>
- <https://medium.com/@abhishekjainindore24/understanding-the-1d-convolutional-layer-in-deep-learning-7a4cb994c981>

